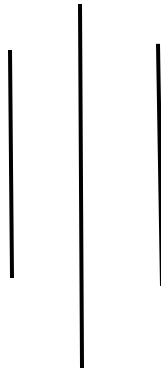




Tribhuvan University
Faculty of Management
Kathmandu, Nepal



DATA MINING
Project Documentation
On
Simple Dataset

Submitted By:
Ritika Koirala

Submitted To:
Mahesh Adhikari

Section A: Data Understanding

This section covers the initial steps required to load, inspect, and understand the dataset prior to any processing or analysis.

i. Importing the Required Libraries

Purpose: Import all necessary Python libraries for data manipulation, visualization, and preprocessing.

```
import pandas as pd
import numpy as np
from matplotlib import pyplot as plt
import seaborn as sns
from sklearn.preprocessing import MinMaxScaler, StandardScaler
from sklearn import preprocessing
```

ii. Loading the Dataset

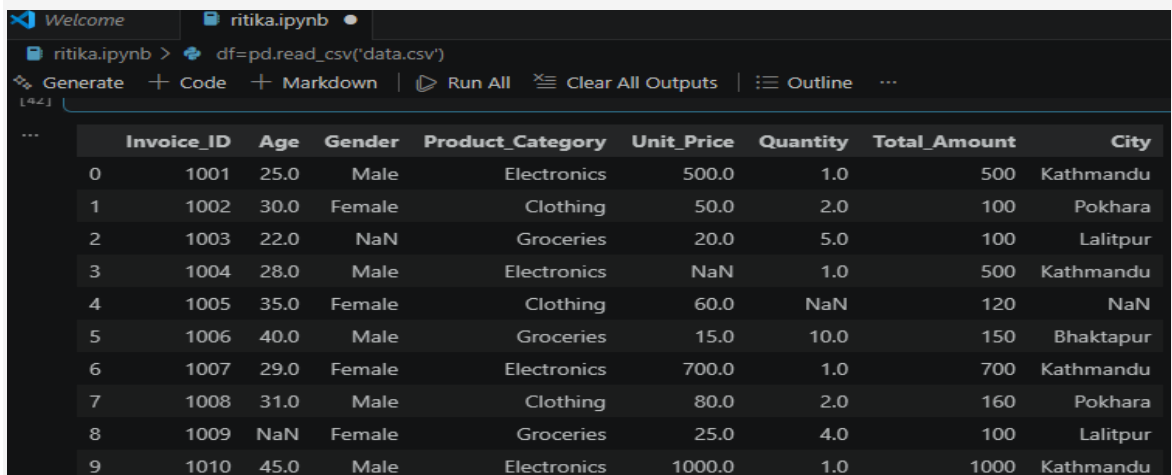
Purpose: Read the dataset from a CSV file into a Pandas DataFrame for further analysis.

```
df = pd.read_csv('data.csv')
```

iii. Displaying the First 10 Rows

Purpose: Preview the initial records of the dataset to understand its structure and contents.

```
df.head(10)
```

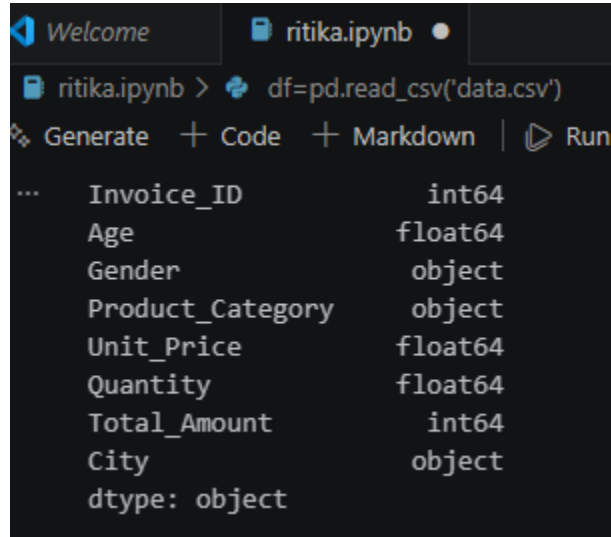


	Invoice_ID	Age	Gender	Product_Category	Unit_Price	Quantity	Total_Amount	City
0	1001	25.0	Male	Electronics	500.0	1.0	500	Kathmandu
1	1002	30.0	Female	Clothing	50.0	2.0	100	Pokhara
2	1003	22.0	NaN	Groceries	20.0	5.0	100	Lalitpur
3	1004	28.0	Male	Electronics	NaN	1.0	500	Kathmandu
4	1005	35.0	Female	Clothing	60.0	NaN	120	NaN
5	1006	40.0	Male	Groceries	15.0	10.0	150	Bhaktapur
6	1007	29.0	Female	Electronics	700.0	1.0	700	Kathmandu
7	1008	31.0	Male	Clothing	80.0	2.0	160	Pokhara
8	1009	NaN	Female	Groceries	25.0	4.0	100	Lalitpur
9	1010	45.0	Male	Electronics	1000.0	1.0	1000	Kathmandu

iv. Checking Data Types of Each Column

Purpose: Verify the data type assigned to each column to identify any type mismatches requiring correction.

```
df.dtypes
```

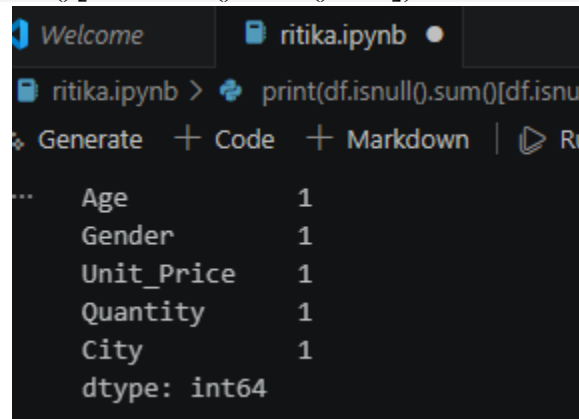
A screenshot of a Jupyter Notebook interface. The top bar shows a 'Welcome' tab and a file named 'ritika.ipynb'. Below the bar, the code cell contains 'df=pd.read_csv('data.csv')'. The output area shows a table of data types for each column: Invoice_ID (int64), Age (float64), Gender (object), Product_Category (object), Unit_Price (float64), Quantity (float64), Total_Amount (int64), City (object), and dtype: object.

```
... Invoice_ID      int64
Age      float64
Gender   object
Product_Category object
Unit_Price float64
Quantity float64
Total_Amount int64
City     object
dtype: object
```

v. Identifying Missing Values

Purpose: Detect columns containing missing values so that appropriate imputation strategies can be applied.

```
print(df.isnull().sum()[df.isnull().sum() > 0])
```

A screenshot of a Jupyter Notebook interface. The top bar shows a 'Welcome' tab and a file named 'ritika.ipynb'. Below the bar, the code cell contains 'print(df.isnull().sum()[df.isnull().sum() > 0])'. The output area shows a table of missing value counts for each column: Age (1), Gender (1), Unit_Price (1), Quantity (1), and City (1). The dtype is int64.

```
... Age      1
Gender  1
Unit_Price  1
Quantity  1
City     1
dtype: int64
```

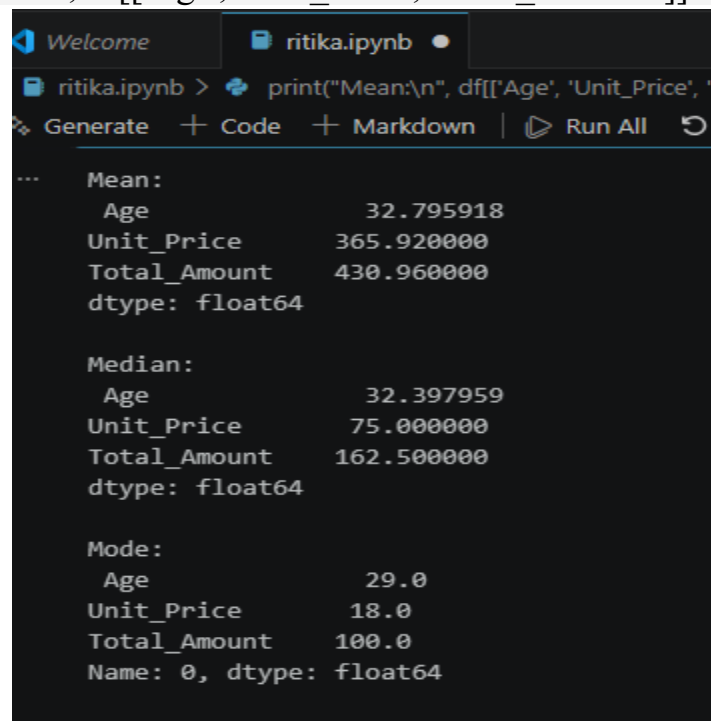
Section B: Missing Data Handling

Missing values in numerical columns are addressed using statistical measures (mean, median, mode) based on the distribution of each column.

1. Compute Descriptive Statistics for Numerical Columns

Purpose: Calculate mean, median, and mode to determine the appropriate imputation strategy for each column.

```
print('Mean:\n', df[['Age', 'Unit_Price', 'Total_Amount']].mean())
print('\nMedian:\n', df[['Age', 'Unit_Price', 'Total_Amount']].median())
print('\nMode:\n', df[['Age', 'Unit_Price', 'Total_Amount']].mode().iloc[0])
```



```
ritika.ipynb > print("Mean:\n", df[['Age', 'Unit_Price', 'Total_Amount']].mean())
Generate + Code + Markdown | Run All

... Mean:
      Age      32.795918
Unit_Price  365.920000
Total_Amount 430.960000
dtype: float64

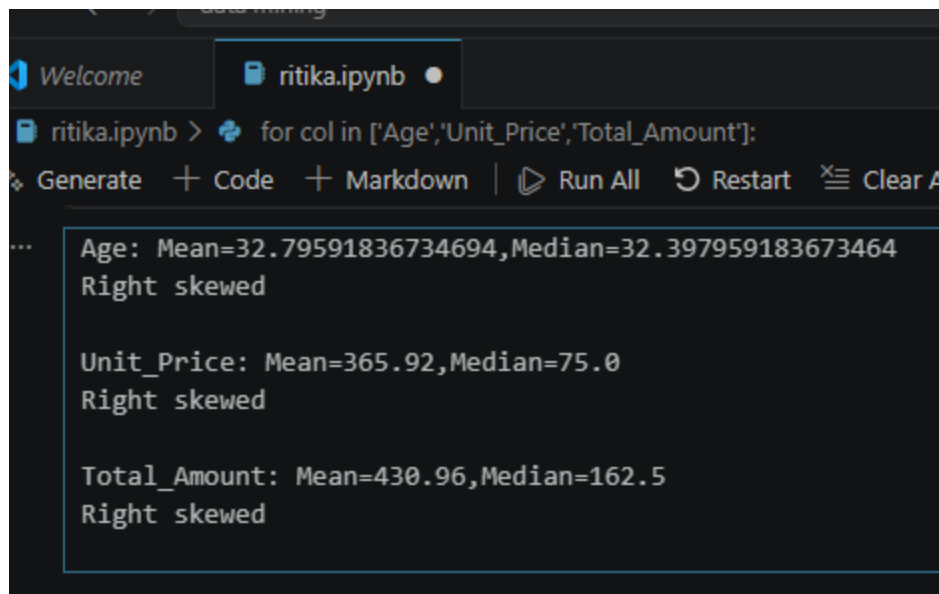
Median:
      Age      32.397959
Unit_Price   75.000000
Total_Amount 162.500000
dtype: float64

Mode:
      Age      29.0
Unit_Price   18.0
Total_Amount 100.0
Name: 0, dtype: float64
```

2. Assess Distribution Skewness

Purpose: Compare mean and median values to determine the direction of skewness and select the optimal imputation method.

```
for col in ['Age', 'Unit_Price', 'Total_Amount']:
    mean = df[col].mean()
    median = df[col].median()
    print(f'{col}: Mean={mean}, Median={median}')
    if mean > median:
        print('Right skewed\n')
    elif mean < median:
        print('Left skewed\n')
    else:
        print('Symmetric\n')
```



```
for col in ['Age','Unit_Price','Total_Amount']:
```

```
Age: Mean=32.79591836734694,Median=32.397959183673464
Right skewed

Unit_Price: Mean=365.92,Median=75.0
Right skewed

Total_Amount: Mean=430.96,Median=162.5
Right skewed
```

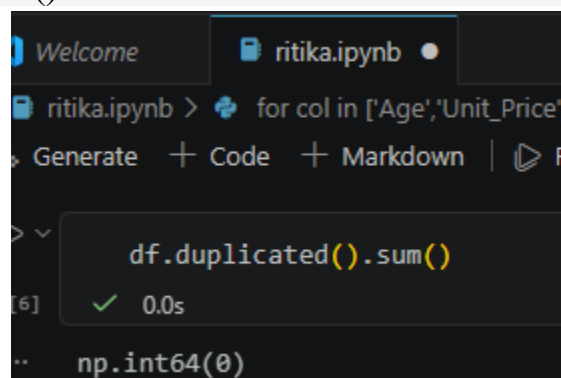
Section C: Data Cleaning

Data cleaning ensures consistency, correctness, and uniformity across the dataset before analysis or modelling.

3. Check and Remove Duplicate Rows

Purpose: Identify and eliminate duplicate records to prevent bias and inaccurate analytical results.

```
df.duplicated().sum()
```



```
df.duplicated().sum()
```

```
[6] ✓ 0.0s
```

```
np.int64(0)
```

4. Ensure Correct Data Types for Numerical Columns

Purpose: Convert columns to numeric format, coercing any non-numeric entries to NaN for subsequent handling.

```
df['Age'] = pd.to_numeric(df['Age'], errors='coerce')
df['Unit_Price'] = pd.to_numeric(df['Unit_Price'], errors='coerce')
```

```
df['Quantity'] = pd.to_numeric(df['Quantity'], errors='coerce')
df['Total_Amount'] = pd.to_numeric(df['Total_Amount'], errors='coerce')
```

5. **Standardize Categorical Values**

Purpose: Convert all categorical text values to lowercase to eliminate inconsistencies caused by case variations.

```
print("\nStandardizing categorical values to lowercase")
df['Gender'] = df['Gender'].str.lower()
df['Product_Category'] = df['Product_Category'].str.lower()
df['City'] = df['City'].str.lower()
```

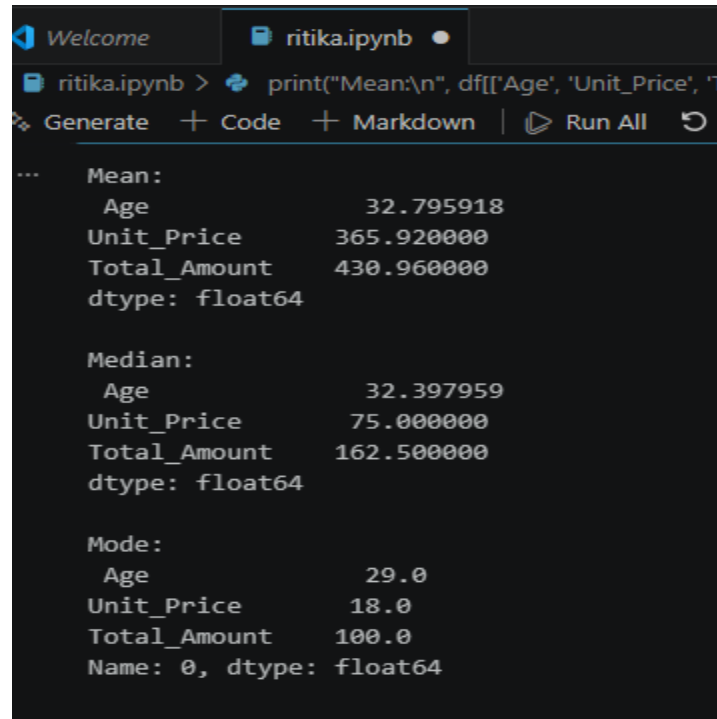
Section D: Statistical Measures

Descriptive statistics are computed for key numerical columns to summarize the central tendency and distribution of the data.

6. **Calculate Mean, Median, and Mode**

Purpose: Compute three measures of central tendency for Age, Unit_Price, and Total_Amount.

```
print('Mean:\n', df[['Age', 'Unit_Price', 'Total_Amount']].mean())
print('\nMedian:\n', df[['Age', 'Unit_Price', 'Total_Amount']].median())
print('\nMode:\n', df[['Age', 'Unit_Price', 'Total_Amount']].mode().iloc[0])
```



```
ritika.ipynb > print("Mean:\n", df[['Age', 'Unit_Price', 'Total_Amount']].mean())
```

```
... Mean:
      Age      32.795918
Unit_Price  365.920000
Total_Amount 430.960000
dtype: float64

Median:
      Age      32.397959
Unit_Price  75.000000
Total_Amount 162.500000
dtype: float64

Mode:
      Age      29.0
Unit_Price  18.0
Total_Amount 100.0
Name: 0, dtype: float64
```

7. Compare Mean vs. Median to Identify Skewness

Purpose: Determine the distributional skewness of each numerical column to inform preprocessing decisions.

```
for col in ['Age', 'Unit_Price', 'Total_Amount']:
    mean = df[col].mean()
    median = df[col].median()
    print(f'{col}: Mean={mean}, Median={median}')
    if mean > median:
        print('Right skewed\n')
    elif mean < median:
        print('Left skewed\n')
    else:
        print('Symmetric\n')
```

```
Welcome ritika.ipynb
ritika.ipynb > for col in ['Age','Unit_Price','Total_Amount']:
Generate + Code + Markdown | Run All Restart
... Age: Mean=32.79591836734694,Median=32.397959183
Right skewed

Unit_Price: Mean=365.92,Median=75.0
Right skewed

Total_Amount: Mean=430.96,Median=162.5
Right skewed
```

Section E: Outlier Detection

Outliers are identified using the Interquartile Range (IQR) method to detect extreme values that may distort analytical results.

8. Apply the IQR Method to Detect Outliers

Purpose: Compute Q1, Q3, and IQR to define acceptable value boundaries, then flag records falling outside those bounds.

```
Q1 = df['Total_Amount'].quantile(0.25)
Q3 = df['Total_Amount'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
print('Lower Bound:', lower_bound)
print('Upper Bound:', upper_bound)

# Detect outliers
outliers = df[(df['Total_Amount'] < lower_bound) | (df['Total_Amount'] >
upper_bound)]
print(outliers)
```

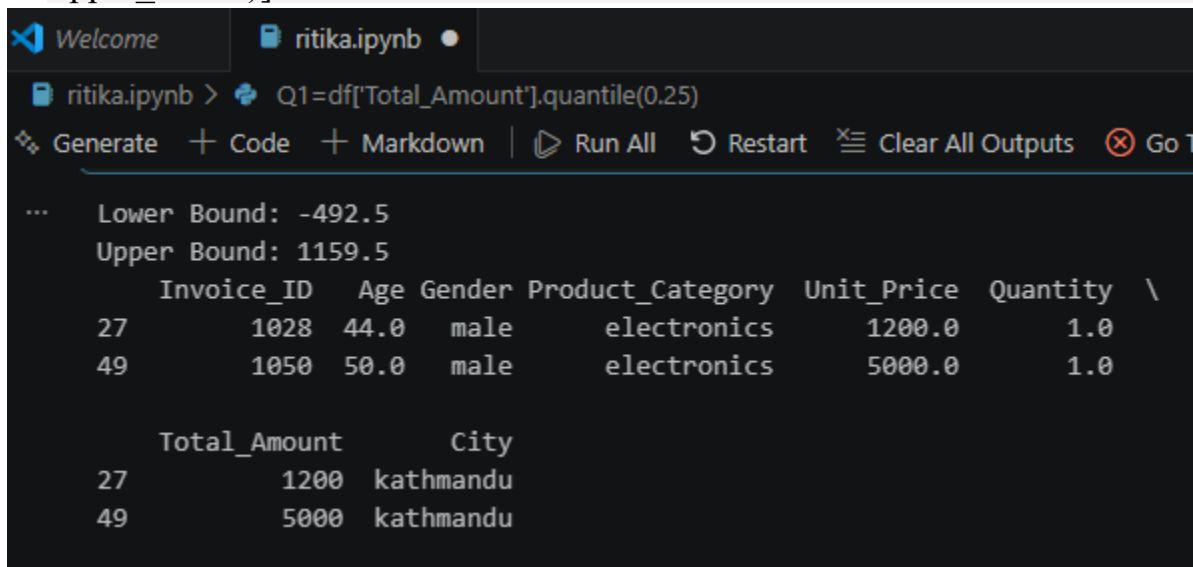

Section F: Outlier Treatment

Detected outliers are removed from the dataset to ensure that subsequent analysis and model training are conducted on reliable, representative data.

9. Remove Records Outside the Acceptable Range

Purpose: Filter out all observations where Total_Amount falls below the lower bound or exceeds the upper bound.

```
df = df[(df['Total_Amount'] >= lower_bound) & (df['Total_Amount'] <= upper_bound)]
```

A screenshot of a Jupyter Notebook interface. The top bar shows 'Welcome' and 'ritika.ipynb'. Below the toolbar, the code cell contains 'Q1=df['Total_Amount'].quantile(0.25)'. The output area displays the calculated bounds: 'Lower Bound: -492.5' and 'Upper Bound: 1159.5'. Below this, a table of data points is shown, including Invoice_ID, Age, Gender, Product_Category, Unit_Price, Quantity, Total_Amount, and City. Two rows are highlighted: one with Invoice_ID 27 and Total_Amount 1200, and another with Invoice_ID 49 and Total_Amount 5000, both from Kathmandu.

	Invoice_ID	Age	Gender	Product_Category	Unit_Price	Quantity	Total_Amount	City
27	1028	44.0	male	electronics	1200.0	1.0	1200	kathmandu
49	1050	50.0	male	electronics	5000.0	1.0	5000	kathmandu

Section G: Feature Scaling

Feature scaling normalizes the range of numerical variables to ensure that no single feature disproportionately influences model performance.

10. Apply Min-Max Normalization

Purpose: Scale Unit_Price, Quantity, and Total_Amount to a uniform [0, 1] range using Min-Max Scaling.

```
scaler = MinMaxScaler()
df[['Unit_Price', 'Quantity', 'Total_Amount']] = scaler.fit_transform(
    df[['Unit_Price', 'Quantity', 'Total_Amount']]
)
```

11. Apply Z-Score Standardization

Purpose: Standardize the Age column using Z-score normalization to achieve a mean of 0 and a standard deviation of 1.

```
scaler_std = StandardScaler()
df['Age'] = scaler_std.fit_transform(df[['Age']])
```

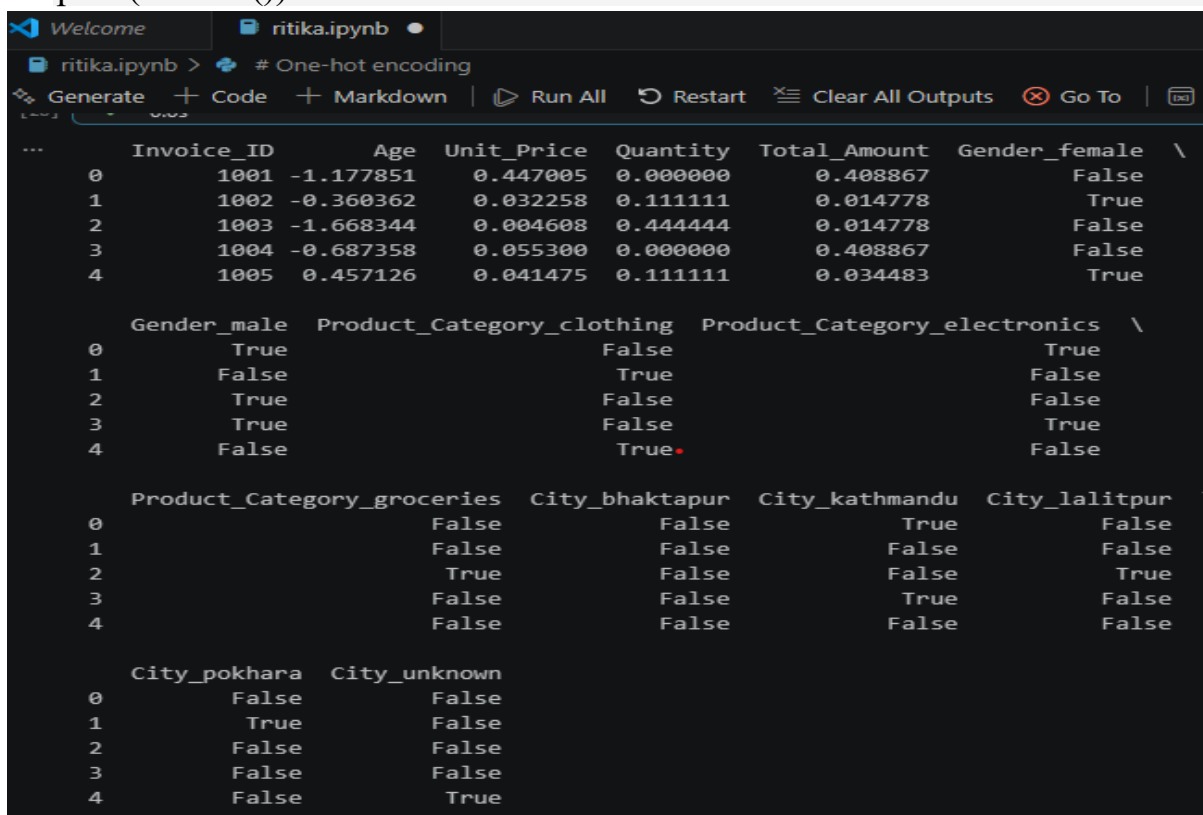
Section H: Categorical Encoding

Categorical variables must be converted to numerical representations to be compatible with machine learning algorithms.

12. Apply One-Hot Encoding to Categorical Columns

Purpose: Convert Gender, Product_Category, and City into binary indicator columns using One-Hot Encoding.

```
df = pd.get_dummies(df, columns=['Gender', 'Product_Category', 'City'])
print(df.head())
```



	Invoice_ID	Age	Unit_Price	Quantity	Total_Amount	Gender_female	Gender_male	Product_Category_clothing	Product_Category_electronics	Product_Category_groceries	City_bhaktapur	City_kathmandu	City_lalitpur	City_pokhara	City_unknown
0	1001	-1.177851	0.447005	0.000000	0.408867	False	True	False	True	False	False	True	False	False	False
1	1002	-0.360362	0.032258	0.111111	0.014778	True	False	True	False	False	False	False	False	True	False
2	1003	-1.668344	0.004608	0.444444	0.014778	False	True	False	False	True	False	False	True	False	False
3	1004	-0.687358	0.055300	0.000000	0.408867	False	True	False	True	False	False	True	False	False	False
4	1005	0.457126	0.041475	0.111111	0.034483	True	False	True	False	False	False	False	False	False	True

Section I: Final Output

The fully processed and cleaned dataset is exported for use in downstream modelling, analysis, or reporting tasks.

13. Export the Cleaned Dataset

Purpose: Save the final preprocessed DataFrame as a CSV file for future use.

